جلسهٔ ۱ — تصویرِ دیجیتال و خطِ لولهٔ پردازش

جزوهٔ درس — هفتهٔ اول

مکملِ اسلایدهای انگلیسیِ جلسهٔ ۱. هدف: فهمِ اینکه یک تصویرِ دیجیتال واقعاً چیست، چطور در کد نمایش داده می‌شود، و کلِ کورس چه مسیری را طی می‌کند.

فهرست مطالب

۱ مقدمه و مقصدِ کورس ۱

۱.۱ چرا پردازش تصویر؟ . . . ۱

۲.۱ راه‌اندازیِ محیط . . . ۲

۳.۱ خطِ لولهٔ پردازش تصویر . . . ۲

۲ تصویرِ دیجیتال چیست؟ ۲

۱.۲ تصویر، یک تابعِ نمونه‌برداری‌شده روی یک شبکه است . . . ۲

۲.۲ یک تصویرِ واقعی از دیتاست . . . ۲

۳.۲ پیکسل‌ها فقط عددند . . . ۳

۴.۲ نمونه‌برداری: چه‌قدر دقیق مکان را می‌سنجیم . . . ۳

۵.۲ کوانتش: چه‌قدر دقیق روشنایی را می‌سنجیم . . . ۳

۶.۲ وضوح، اندازه، و حافظه . . . ۴

۳ رنگ ۴

۱.۳ رنگ: سه عدد به‌ازای هر پیکسل . . . ۴

۲.۳ چرا به خاکستری تبدیل کنیم . . . ۴

۳.۳ فضاهای رنگیِ دیگر و چرایی . . . ۴

۴ تصاویر در کد ۵

۱.۴ تصویر یک آرایهٔ NumPy است . . . ۵
۲.۴ پیکسل، برش، و عملیاتِ برداری . . . ۵
۳.۴ نمایش و اشکال‌زداییِ تصویر . . . ۵
۴.۴ دامِ سرریزِ uint۸ . . . ۶

۵ یک مثالِ عددیِ کامل: خاکستری‌سازی ۶

۱.۵ وضوح، DPI، و معنای عملی . . . ۷

۶ عملیاتِ حسابیِ تصویر — پیش‌درآمد ۷

۷ تمرینِ کلاسیِ وسط‌جلسه (۱۰ دقیقه) ۷

۸ فرمت‌ها و ذخیره‌سازی ۸

۱.۸ عمقِ بیت و بازهٔ دینامیکی . . . ۸

۹ بخشِ عمیق: مدلِ تصویربرداری ۸

۱.۹ نظریهٔ نمونه‌برداری و آلیاسینگ . . . ۸
۲.۹ گاما: پیکسل‌ها نورِ خطی نیستند . . . ۹
۳.۹ پیکسل‌ها از کجا می‌آیند . . . ۹
۴.۹ نویز فیزیکی است، نه باگ . . . ۹
۵.۹ صفحاتِ بیتی (Bit planes) . . . ۹
۶.۹ هیستوگرام — نگاهی اجمالی (پیش‌درآمدِ جلسهٔ ۲) . . . ۱۰
۷.۹ مختصات و قراردادها . . . ۱۰

۱۰ جمع‌بندی ۱۰

۱ مقدمه و مقصدِ کورس

تا پایانِ کورس، شما **DocScan** را می‌سازید: یک عکس از یک سند (صفحه، فرم، رسید) که با زاویه و نورِ ناهموار گرفته شده → یک اسکنِ تمیز، صاف‌شده و قابلِ‌خواندنِ ماشینی، و سپس خواندنِ بخشی از محتوا با یک شبکهٔ عصبیِ کوچک. ابزارها: Python + OpenCV + NumPy. جلسات ۱ تا ۱۱ پردازشِ تصویرِ کلاسیک، و جلسات ۱۲ تا ۱۴ مقدمهٔ یادگیریِ عمیق برای بینایی.

نکتهٔ کلیدی

عادتی که می‌سازیم: اول پیکسل‌ها و ریاضی را بفهم؛ دوم کتابخانه را صدا بزن؛ و سراغِ یک شبکهٔ عصبی فقط وقتی برو که روش‌های کلاسیک کم بیاورند. این ترتیب، تفاوتِ کسی که «تابع صدا می‌زند» و مهندسی که «می‌داند چرا» را می‌سازد.

۱.۱ چرا پردازش تصویر؟

پردازش تصویر همه‌جا هست: تشخیصِ پزشکی (تومور در MRI، خودروی خودران (تشخیصِ خطوط و عابر)، کنترلِ کیفیتِ صنعتی (تشخیصِ عیب)، امنیت (تشخیصِ چهره)، کشاورزی (سلامتِ محصول از تصویرِ هوایی)، و اسناد (OCR، همان DocScan ما). در همهٔ این‌ها، مسئله یکی است: از پیکسل‌های خام به یک تصمیمِ معنادار رسیدن. تکنیک‌های این کورس، بلوک‌های سازندهٔ همهٔ این کاربردهاست.

یادداشت

پردازش تصویرِ کلاسیک (که تا جلسهٔ ۱۱ می‌آموزیم) هنوز حیاتی است، حتی در عصرِ یادگیریِ عمیق: سریع، قابلِ‌تفسیر، بدونِ نیاز به دادهٔ آموزشیِ عظیم، و اغلب پیش‌پردازشِ لازم پیش از یک شبکهٔ عصبی. بهترین سیستم‌ها کلاسیک و عمیق را ترکیب می‌کنند — دقیقاً کاری که DocScan می‌کند.

۲.۱ راه‌اندازیِ محیط

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install numpy opencv-python scikit-image matplotlib
# deep-learning weeks (12-14):
pip install torch torchvision
```

NumPy موتورِ آرایه است؛ OpenCV کتابخانهٔ اصلیِ پردازش تصویر؛ scikit-image مکملِ پایتونیک‌تر؛ Matplotlib برای نمایش. همه روی نسخه‌های سازگار باشند تا کدها یکسان اجرا شوند.

۳.۱ خطِ لولهٔ پردازش تصویر

تقریباً هر تسک در این کورس مسیری در این خطِ لوله است:

تصمیم ← استخراجِ ویژگی ← قطعه‌بندی ← بهبود ← پیش‌پردازش ← اکتساب

DocScan همهٔ این مسیر را می‌پیماید: از عکسِ خام تا خواندنِ محتوا.

۲ تصویرِ دیجیتال چیست؟

۱.۲ تصویر، یک تابعِ نمونه‌برداری‌شده روی یک شبکه است

یک تصویرِ پیوسته یک تابعِ $f(x, y)$ است: روشناییِ نقطه‌ای در موقعیتِ (x, y). یک تصویرِ **دیجیتال** این تابع را روی یک شبکه نمونه‌برداری می‌کند و هر مقدار را گِرد می‌کند؛ نتیجه **پیکسل**‌ها هستند. یک تصویرِ خاکستری یک آرایهٔ دوبعدی است؛ یک تصویرِ رنگی سه آرایهٔ این‌چنینی (کانال).

ریاضیات

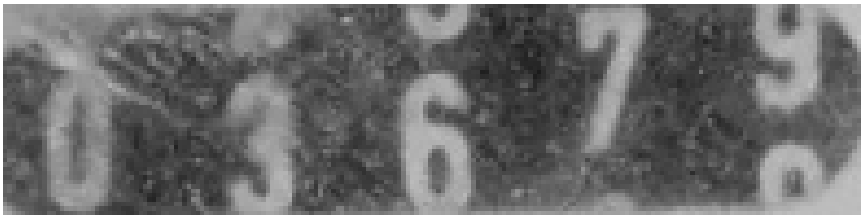
$f(x, y)$ با $x = ۰, \dots, W-۱$ و $y = ۰, \dots, H-۱$، و هر مقدار در مجموعهٔ $\{۰, \dots, ۲۵۵\}$ (برای ۸ بیت). صفر = سیاه، ۲۵۵ = سفید، بینابین = خاکستری.

۲.۲ یک تصویرِ واقعی از دیتاست

مثالِ درجریانِ ما یک نمای شمارندهٔ کنتورِ آب است (از دیتاستِ شما، E:/data/abfa_image/numerators). خواندنِ همین ارقام، دقیقاً هدفِ پروژهٔ نهاییِ DocScan است.



تصویرِ اصلیِ رنگی — یک نمای رقمِ کنتور (۱۹۶ × ۴۸ پیکسل)



نسخهٔ خاکستری — یک کانال، همان ساختار

۳.۲ پیکسل‌ها فقط عددند

«پردازشِ» یک تصویر یعنی انجامِ محاسبات روی این اعداد. هیچ جادویی وجود ندارد: هر عملیات، ریاضی روی یک آرایه است. این جملهٔ ساده، کلِ فلسفهٔ کورس است — وقتی یک فیلتر، یک آستانه، یا یک تشخیصِ لبه را می‌بینید، در پسِ آن فقط جمع، ضرب، و مقایسهٔ اعداد است.

۴.۲ نمونه‌برداری: چه‌قدر دقیق مکان را می‌سنجیم

نمونه‌برداری (sampling) = انتخابِ نقاطِ شبکه (وضوح، resolution). نمونه‌های خیلی کم یعنی ازدست‌رفتنِ جزئیات — یا بدتر، جعلِ آن (aliasing، در بخشِ عمیق). پیکسل‌های بیشتر = جزئیاتِ مکانیِ بیشتر و دادهٔ بیشتر.

۵.۲ کوانتش: چه‌قدر دقیق روشنایی را می‌سنجیم

کوانتش (quantization) = گِردکردنِ هر نمونه به مجموعهٔ متناهی از سطوح (عمقِ بیت، bit depth). ۸ بیت = ۲۵۶ سطح (۰ تا ۲۵۵)؛ ۱ بیت = سیاه/سفید؛ ۱۶ بیت = ۶۵۵۳۶ سطح (پزشکی/RAW). سطوحِ خیلی کم → **نوارنوارشدنِ** (banding) دیدنی در گرادیان‌های نرم.

یادداشت

وضوح (نمونه‌برداری) و عمقِ بیت (کوانتش) دو محورِ «چه‌قدر تصویر داریم» هستند: یکی دقتِ مکانی، دیگری دقتِ روشنایی. هر دو حجمِ داده و کیفیت را تعیین می‌کنند.

۶.۲ وضوح، اندازه، و حافظه

یک تصویرِ خاکستریِ ۸ بیتیِ $W \times H$ برابرِ $W \cdot H$ بایت است؛ رنگی سه برابر. یک عکسِ رنگیِ ۱۲ مگاپیکسلی حدودِ ۳۶ مگابایت فشرده‌نشده است. برای همین فرمت‌ها فشرده می‌کنند — و برای همین پردازشِ تصاویرِ بزرگ به دقت نیاز دارد (جلسهٔ ۱۱). پیش از هر کاری شکل و نوعِ دادهٔ تصویر را بدانید: img.shape و img.dtype.

۳ رنگ

۱.۳ رنگ: سه عدد به‌ازای هر پیکسل

RGB: کانال‌های قرمز، سبز، آبی؛ همان‌طور که نمایشگرها نور می‌تابانند. یک پیکسلِ رنگی یک سه‌تایی (R, G, B) است، هرکدام ۰ تا ۲۵۵. خاکستری‌سازی رنگ را به یک روشنایی فرومی‌کاهد — اغلب اولین گام (دادهٔ کمتر، ساختارِ یکسان).

ریاضیات

روشنایی (luminance): $Y = ۰٫۲۹۹R + ۰٫۵۸۷G + ۰٫۱۱۴B$. سبز غالب است چون چشمِ ما به سبز حساس‌تر است. این وزن‌دهیِ ادراکی، از میانگینِ سادهٔ $(R+G+B)/۳$ درست‌تر است.

۲.۳ چرا به خاکستری تبدیل کنیم

بسیاری از عملیات (لبه، آستانه) به روشنایی نیاز دارند، نه رنگ — یک کانال، یک‌سومِ کار. DocScan عمدتاً در خاکستری کار می‌کند؛ رنگ فقط وقتی برمی‌گردد که اطلاعات حمل کند. اما خاکستری‌سازی اطلاعات را دور می‌ریزد — اگر رنگ مهم است (مثلاً قطعه‌بندیِ رنگیِ جلسهٔ ۸) این کار را نکنید.

۳.۳ فضاهای رنگیِ دیگر و چرایی

فضا	کانال‌ها	مناسبِ
RGB	قرمز، سبز، آبی	ذخیره، نمایش
خاکستری	روشنایی	بیشترِ پردازشِ کلاسیک
HSV	رنگ‌مایه، اشباع، مقدار	انتخابِ رنگ‌محور (مقاوم به نور)
Lab	روشنایی، a، b	فاصلهٔ ادراکی، تصحیحِ رنگ

نکتهٔ کلیدی

انتخابِ فضای رنگیِ درست می‌تواند یک مسئلهٔ سخت را آسان کند — مثلاً «شیءِ قرمز را پیدا کن» در HSV بدیهی است و در RGB دردناک. در HSV، رنگ‌مایه (Hue) رنگ را از روشنایی جدا می‌کند، پس انتخاب در برابرِ سایه و نور مقاوم می‌شود.

۴ تصاویر در کد

۱.۴ تصویر یک آرایهٔ NumPy است

```
import cv2, numpy as np
img = cv2.imread('page.jpg')          # BGR, shape (H, W, 3), dtype uint8
print(img.shape, img.dtype)           # e.g. (1200, 900, 3) uint8
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # (H, W)
print(gray[0, 0])                     # a single pixel's intensity, 255..0
```

هشدار / دام

دامِ OpenCV: تصاویر را به‌صورتِ **BGR** می‌خواند، نه RGB. فراموش‌کردنِ این، قرمز و آبی را در همه‌جا جابه‌جا می‌کند. پیش از plt.imshow حتماً BGR2RGB کنید.

۲.۴ پیکسل، برش، و عملیاتِ برداری

```
h, w = gray.shape
roi = gray[100:200, 50:150]            # a rectangular region (a view)
gray[gray < 50] = 0                     # vectorized: dark pixels -> black
brighter = np.clip(gray.astype(int) + 40, 0, 255).astype(np.uint8)
```

هشدار / دام

هرگز در پایتون روی پیکسل‌ها حلقه نزنید — صد برابر کندتر است. از برداری‌سازیِ NumPy یا خودِ OpenCV استفاده کنید. این یکی از مهم‌ترین عادت‌های پردازشِ تصویرِ کارآمد است.

۳.۴ نمایش و اشکال‌زداییِ تصویر

پردازش تصویر بصری است؛ همیشه نتیجهٔ هر گام را ببینید. ابزارها:

```
import matplotlib.pyplot as plt
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.axis('off'); plt.show()
plt.imshow(gray, cmap='gray', vmin=0, vmax=255)   # grayscale: set the range!
cv2.imwrite('gray.png', gray)                      # PNG = lossless
```

نکتهٔ ظریف: هنگامِ نمایشِ خاکستری با Matplotlib، اگر vmin/vmax را تنظیم نکنید، مقادیر به‌طورِ خودکار کشیده می‌شوند و ممکن است تصویری کم‌کنتراست را پرکنتراست نشان دهد در حالی که داده تغییر نکرده. برای دیدنِ واقعیتِ داده، بازه را صریح ۰ تا ۲۵۵ بگذارید. این تفاوتِ «چیزی که می‌بینید» و «داده‌ای که دارید» یک منبعِ رایجِ سردرگمی است.

یادداشت

عادتِ خوب: پس از هر عملیات، img.min()، img.max()، و img.dtype را چاپ کنید. اگر یک تصویر که باید ۰ تا ۲۵۵ باشد ناگهان ۰ تا ۱ یا اعشاری شد، یک خطا رخ داده. این چند خطِ اشکال‌زدایی ساعت‌ها گیجی را صرفه‌جویی می‌کند.

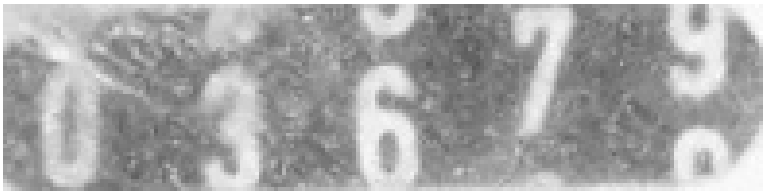
۴.۴ دامِ سرریزِ uint۸

محاسبات روی uint8 **دور می‌زند** (wrap around): ۱۰۰ + ۲۰۰ = ۴۴، نه ۲۵۵! به float/int تبدیل کنید، محاسبه کنید، به ۰ تا ۲۵۵ clip کنید، و برگردانید. یا از عملیاتِ اشباعیِ OpenCV (cv2.add) که برایتان محدود می‌کند استفاده کنید.

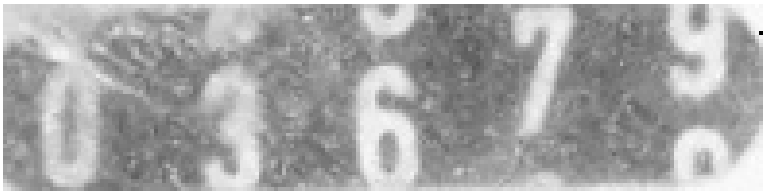
نکتهٔ کلیدی

یک «روشن‌سازی» که نواحیِ روشن را تاریک می‌کند، تقریباً همیشه سرریزِ uint8 است. این یکی از اولین باگ‌هایی است که هر تازه‌کاری تجربه می‌کند.

روی همان تصویرِ کنتور، تفاوت را ببینید:



$+60$ درست (با clip): یکنواخت روشن‌تر



$+60$ در uint8: پیکسل‌های روشن دور زدند و تاریک شدند

۵ یک مثالِ عددیِ کامل: خاکستری‌سازی

بیایید تبدیلِ یک پیکسلِ رنگی به خاکستری را دستی انجام دهیم تا فرمول ملموس شود. فرض کنید پیکسلی داریم با $(R, G, B) = (۱۰۰, ۲۰۰, ۵۰)$ (یک سبزِ روشن).

میانگینِ ساده: $(۱۰۰ + ۲۰۰ + ۵۰)/۳ = ۱۱۶٫۶۷ \approx ۱۱۷$.

روشناییِ ادراکی: $Y = ۰٫۲۹۹(۱۰۰) + ۰٫۵۸۷(۲۰۰) + ۰٫۱۱۴(۵۰) = ۲۹٫۹ + ۱۱۷٫۴ + ۵٫۷ = ۱۵۳$.

تفاوت چشمگیر است: میانگینِ ساده ۱۱۷ می‌دهد، اما فرمولِ ادراکی ۱۵۳ — چون سبز (که در این پیکسل غالب است) در ادراکِ انسان وزنِ بیشتری دارد. برای همین یک شیءِ سبز با میانگینِ ساده تیره‌تر از آنچه باید به‌نظر می‌رسد.

نکتهٔ کلیدی

این تفاوت در عمل مهم است: اگر بعداً روی تصویرِ خاکستری آستانه بگذارید (جلسهٔ ۸)، انتخابِ فرمولِ خاکستری‌سازی می‌تواند نتیجه را عوض کند. کتابخانه‌ها (OpenCV) فرمولِ ادراکی را استفاده می‌کنند؛ بدانید چرا.

۱.۵ وضوح، DPI، و معنای عملی

وضوح فقط «تعدادِ پیکسل» نیست؛ نسبتِ آن به اندازهٔ فیزیکی هم مهم است. **DPI** (نقطه در اینچ) می‌گوید چند پیکسل به‌ازای هر اینچ داریم. برای DocScan این حیاتی است: یک سند که با DPI پایین اسکن شود، متنِ ریز را از دست می‌دهد و OCR شکست

می‌خورد. قاعدهٔ عملی: برای خواندنِ متن، حداقل ۳۰۰ DPI. یک عکسِ موبایلی از فاصلهٔ زیاد ممکن است پیکسل‌های کافی روی هر حرف نداشته باشد — که یکی از سخت‌ترین چالش‌های پروژهٔ شماست.

۶ عملیاتِ حسابیِ تصویر — پیش‌درآمد

بسیاری از پردازش‌ها ترکیبی از عملیاتِ حسابیِ ساده روی آرایه‌ها هستند:

- **جمع/تفریق**: روشن/تاریک‌کردن، یا تفاوتِ دو تصویر (تشخیصِ تغییر، حذفِ پس‌زمینه).
- **ضرب**: اعمالِ یک ماسک، یا تنظیمِ کنتراست.
- **ترکیب** (blend): $\alpha \cdot A + (1-\alpha) \cdot B$ برای محوِ تدریجیِ دو تصویر.

مثالِ کاربردی: تفریقِ دو فریمِ متوالیِ یک دوربینِ ثابت، پیکسل‌هایی را که تغییر کرده‌اند (حرکت) برجسته می‌کند — پایهٔ تشخیصِ حرکت. همهٔ این‌ها فقط جمع و ضربِ آرایه‌اند، با مراقبت از سرریز.

```
diff = cv2.absdiff(frame1, frame2)     # |frame1 - frame2|, saturating
blended = cv2.addWeighted(a, 7.0, b, 3.0, 0)  # *7.0a + *3.0b
```

۷ تمرینِ کلاسیِ وسط‌جلسه (۱۰ دقیقه)

تمرین

پیش‌بینی کنید، بعد اجرا کنید:

```
gray = np.array([[100, 150],[200, 250]], dtype=np.uint8)
a = gray + 60          # (1) what are the four values?
b = cv2.add(gray, 60)  # (2) and here?
c = gray.mean()        # (3) what does this represent?
```

(۱) a را دستی حساب کنید — مراقبِ سرریز. (۲) b چه فرقی دارد؟ (۳) c چیست، و یک c خیلی پایین چه چیزی دربارهٔ تصویر می‌گوید؟

جمع‌بندی: $a = [[160,210],[4,54]]$ — ۶۰+۲۰۰ و ۶۰+۲۵۰ دور زدند (سرریز). $b = [[160,210],[255,255]]$ — cv2.add اشباع می‌کند (به ۲۵۵ محدود). c میانگینِ روشنایی است ($\approx$ ۱۷۵)؛ میانگینِ خیلی پایین یعنی تصویرِ عمدتاً تاریک (کم‌نور). میانگین/هیستوگرامِ یک تصویر پیش از نگاه‌کردن، دربارهٔ نوردهی به شما می‌گوید — جلسهٔ ۲.

۸ فرمت‌ها و ذخیره‌سازی

فرمت	فشرده‌سازی	کاربرد
PNG	بی‌اتلاف	ماسک، خطوط، نتایجِ میانی
JPEG	بااتلاف	عکس؛ فایلِ کوچک ولی مصنوعات
TIFF	بی‌اتلاف/بدون	بایگانی، علمی، چندصفحه
BMP/RAW	بدون	سادگی / دادهٔ حسگر

هشدار / دام

هرگز نتایجِ میانی را JPEG ذخیره نکنید — مصنوعاتش پردازشِ بعدی را خراب می‌کنند. JPEG جزئیاتِ فرکانس‌بالا را در بلوک‌های ۸ × ۸ دور می‌ریزد؛ نامرئی برای چشم، سم برای تشخیصِ لبه. تا آخرین مرحله PNG استفاده کنید. ذخیرهٔ مکررِ یک JPEG آسیب را انباشته می‌کند.

۱.۸ عمقِ بیت و بازهٔ دینامیکی

۸ بیت برای نمایش خوب است؛ حسگرهای ۱۲ تا ۱۶ بیتی جزئیاتِ سایه/های‌لایت را نگه می‌دارند. **بازهٔ دینامیکی** = نسبتِ روشن‌ترین به تاریک‌ترین که حسگر می‌تواند ثبت کند. HDR چند نوردهی را ترکیب می‌کند تا از بازهٔ یک حسگر فراتر رود.

۹ بخشِ عمیق: مدلِ تصویربرداری

۱.۹ نظریهٔ نمونه‌برداری و آلیاسینگ

نایکوئیست (Nyquist): برای ثبتِ جزئیات در فرکانسِ f، با نرخِ بیش از $2f$ نمونه‌برداری کنید. اگر کمتر نمونه بگیرید، فرکانس‌های بالا خود را به‌جای فرکانس‌های پایین جا می‌زنند. این **آلیاسینگ** است: الگوهای مواره (moiré)، لبه‌های دندانه‌دار، چرخِ گاری که به عقب می‌چرخد. راه‌حل: پیش از کاهشِ اندازه، تصویر را تار (پایین‌گذر) کنید — جزئیاتی را که نمی‌توانید نمایش دهید حذف کنید (جلسات ۳ و ۶).

هشدار / دام

کوچک‌کردنِ یک تصویر بدونِ تارکردنِ اول، مصنوعاتِ آلیاسینگ می‌سازد که هیچ گامِ بعدی نمی‌تواند جبرانشان کند. این یک اشتباهِ رایج و برگشت‌ناپذیر است.

۲.۹ گاما: پیکسل‌ها نورِ خطی نیستند

نمایشگرها و فایل‌ها **گاما–کدشده**‌اند: مقدارِ پیکسل $\neq$ شدتِ نورِ خطی. ذخیرهٔ ادراکی (کدهای بیشتر برای تُن‌های تاریک، جایی که چشم حساس‌تر است) بیت صرفه‌جویی می‌کند. برای ریاضیِ درست (میانگین‌گیری، ترکیب)، گاهی اول باید خطی‌سازی کنید

— یک باگِ ظریفِ رایج.

ریاضیات

کدشده $\approx$ (خطی)$^{1/\gamma}$ با $\gamma \approx 2{,}2$. از گاما در جلسهٔ ۲ برای کنترلِ کنتراست استفاده می‌کنیم.

۳.۹ پیکسل‌ها از کجا می‌آیند

فایل $\rightarrow$ دی‌موزائیک + ISP $\rightarrow$ حسگر + بایر $\rightarrow$ لنز $\rightarrow$ صحنه

حسگرهای واقعی یک رنگ به‌ازای هر پیکسل ثبت می‌کنند (موزائیکِ **بایر**، Bayer) و بقیه را درون‌یابی می‌کنند. نویز، تاری، و انحرافِ رنگ اینجا وارد می‌شوند — دقیقاً همان چیزی که پاک‌سازی می‌کنیم (جلسات ۴ و ۲).

۴.۹ نویز فیزیکی است، نه باگ

شمارشِ فوتون تصادفی است $\rightarrow$ نویزِ شات (shot noise)؛ الکترونیک نویزِ خواندن اضافه می‌کند. نورِ کم / ISOی بالا = نویزِ بیشتر. این مادهٔ خامی است که جلسهٔ ۴ حذفش می‌کند. فهمِ اینکه نویز از کجا می‌آید به شما می‌گوید چطور مدل و حذفش کنید.

۵.۹ صفحاتِ بیتی (Bit planes)

یک تصویرِ ۸ بیتی را می‌توان به ۸ «صفحهٔ بیتی» تجزیه کرد: صفحهٔ بیتِ ۷ (پرارزش‌ترین) ساختارِ اصلیِ تصویر را دارد، و صفحهٔ بیتِ ۰ (کم‌ارزش‌ترین) عمدتاً نویز است. این توضیح می‌دهد چرا حذفِ چند بیتِ کم‌ارزش (کوانتشِ درشت‌تر) تصویر را چندان خراب نمی‌کند اما فضا صرفه‌جویی می‌کند — ایده‌ای که پشتِ فشرده‌سازی است. همچنین پایهٔ برخی تکنیک‌های پنهان‌نگاری (steganography) است که داده را در بیت‌های کم‌ارزش مخفی می‌کنند.

۶.۹ هیستوگرام — نگاهی اجمالی (پیش‌درآمدِ جلسهٔ ۲)

هیستوگرام یک تصویر می‌شمارد چند پیکسل هر سطحِ روشنایی را دارند. بدونِ نگاه‌کردن به تصویر، هیستوگرام به شما می‌گوید: آیا تصویر تاریک است (پیک در چپ)، روشن است (پیک در راست)، کم‌کنتراست است (فشرده در وسط)، یا خوش‌کنتراست (پخش‌شده). این اولین ابزارِ تشخیصیِ شماست و جلسهٔ ۲ کاملاً به آن می‌پردازد. همین حالا بدانید: میانگین و هیستوگرام، «امضای» یک تصویرند.

۷.۹ مختصات و قراردادها

تصاویر با $[\text{row, col}] = [\text{y, x}]$ اندیس‌گذاری می‌شوند — عکسِ (x, y) ریاضی. مبدأ گوشهٔ بالا-چپ است؛ y به پایین افزایش می‌یابد. نقاطِ OpenCV به‌صورتِ (x, y)اند اما اندیس‌های NumPy به‌صورتِ [y, x]. این‌ها را جدا نگه دارید.

هشدار / دام

جابه‌جاییِ x و y باگِ شمارهٔ یکِ تازه‌کارها در کدِ تصویر است. وقتی چیزی ترانهاده به‌نظر می‌رسد، اول این را بررسی کنید.

۱۰ جمع‌بندی

- توضیحِ نمونه‌برداری، کوانتش، وضوح، و عمقِ بیت.
- بارگذاریِ تصویر، بازبینیِ شکل/نوع، تبدیلِ رنگ، برشِ نواحی — به‌صورتِ برداری.
- انتخابِ فضای رنگی و فرمت برای یک تسک؛ پرهیز از دام‌های xy/BGR/uint8.
- نام‌بردنِ منشأِ فیزیکیِ نویز، تاری، و آلیاسینگ.

تمرین

تحویلیِ این هفته (پیش از جلسهٔ ۲): (۱) سه تصویر را بارگذاری کنید؛ شکل، نوع، کمینه/بیشینه/میانگین را چاپ کنید؛ به خاکستری و HSV تبدیل کنید. (۲) یک brighten(img, k) درست (بدونِ سرریز) بنویسید و با cv2.add مقایسه کنید. (۳) یک عکس از یک سند با موبایل بگیرید — تصویرِ آزمونِ DocScan شما. ویژگی‌هایش را گزارش دهید. (۴) یادداشتِ کوتاه: برای عکسِ شما، کدام گام‌های خطِ لوله سخت‌ترند و چرا؟

چک‌لیستِ دام‌های رایجِ این هفته

دام	راه‌حل
BGR در برابرِ RGB	پیش از imshow، cv2.COLOR_BGR2RGB
سرریزِ uint8	به int تبدیل، clip، برگرداندن؛ یا cv2.add
جابه‌جاییِ x و y	اندیسِ NumPy برابرِ [y, x] است
حلقه روی پیکسل‌ها	برداری‌سازیِ NumPy
ذخیرهٔ میانی به JPEG	PNG تا آخرین مرحله
میانگینِ سادهٔ رنگ	فرمولِ روشناییِ وزن‌دار

واژه‌نامهٔ کوتاه

اصطلاح	معنی
Pixel	کوچک‌ترین عنصرِ تصویر؛ یک مقدارِ روشنایی/رنگ
Sampling	نمونه‌برداریِ مکانی (وضوح)
Quantization	گِردکردنِ روشنایی به سطوحِ متناهی (عمقِ بیت)
Channel	یکی از آرایه‌های رنگ (مثلاً R، G، B)
Aliasing	جعلِ فرکانس‌بالا در اثرِ کم‌نمونه‌برداری
uint8	نوعِ ۸ بیتیِ بدونِ‌علامت (۰ تا ۲۵۵)؛ مراقبِ سرریز

یک تصویر فقط اعدادی روی یک شبکه است. باقیِ کورس، حساب‌وکتاب روی همان اعداد است.